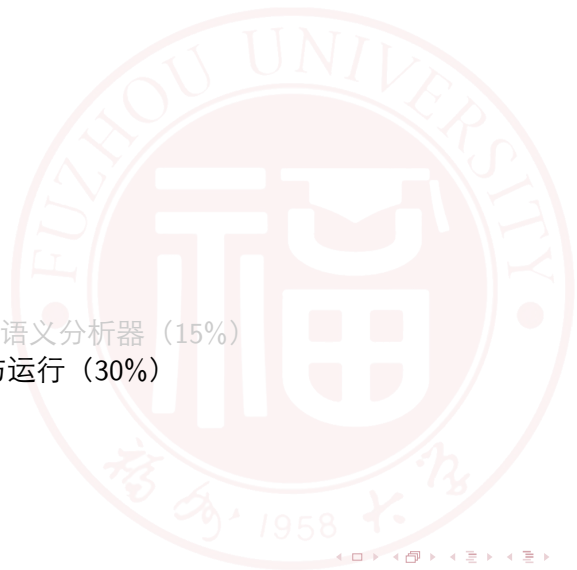




1 简介

2 Mandrill 参考手册

3 实验安排

实验一：词法分析器（15%）

实验二：语法分析器（20%）

实验三：基于 ANTLR 的编译器语义分析器（15%）

实验四：虚拟机汇编代码输出与运行（30%）

报告撰写（20%）

4 结束语

虚拟机汇编代码输出与运行要求

- 从 `stdin` 读入源代码，输出字节码到 `stdout`。
- 部署在评测机上的虚拟机程序将从 `stdin` 读入字节码并执行，输出结果到 `stdout`。
- 输入的源代码保证不存在语法和词法错误。
- 保证源代码在给定输入的情况下不会发生死循环。

Mandrill 虚拟机

- Mandrill 虚拟机采用类似 Java 最初版本的虚拟机架构，即虚拟机主要维护一个栈，用于存储操作数。
- 此外，虚拟机还会维护两个寄存器分别是程序计数器（Program Counter，PC）和栈指针（Stack Pointer，SP）²。
- 程序计数器是一个 32 位整数，指向代码存储区中下一轮执行指令的第一字节地址，从 0x00000000 开始，每条指令占 8 个字节，正常指令执行后 PC 自动加 8。跳转和分支求值指令会修改 PC 的值。
- 当 PC 恰好等于 0xFFFFFFFF 时，虚拟机退出并返回 0。即程序执行的最后一条指令强制为 `jump 0xFFFFFFFF`。

²指向第一个局部变量的地址

虚拟机汇编——数据存取指令（入栈）

Mandrill 虚拟机的指令一般由‘命令+操作对象’构成，其中命令为固定单词，操作对象为立即数、运算符类型以及内存地址构成。

接下来将介绍每一个指令。

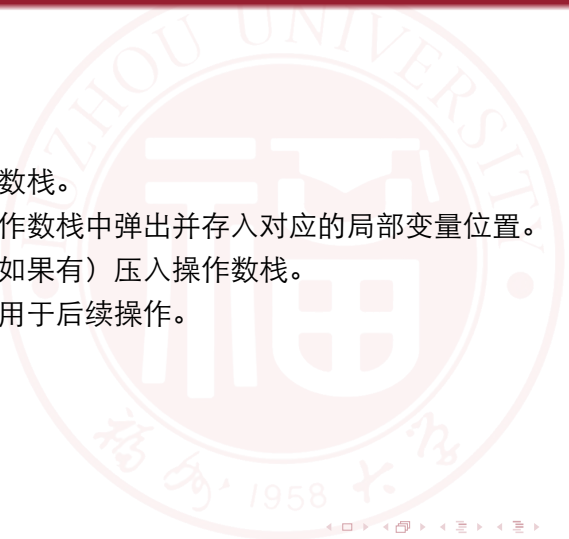
- 全局数据加载指令：用于从数据存储区对应位置读取数据并存入操作数栈。
格式：dload x, x 为变量编号，从 0 开始。
- 局部数据加载指令：用于从数据存储区对应位置读取数据并存入操作数栈。
格式：dlload x, x 为局部变量编号，从 0 开始，是相对于栈指针的偏移量。
- 绝对数据加载指令：从栈顶获取地址并从对应的内存地址中读取数据并存入操作数栈，可用于访问数组元素。
格式：daload x, x 为无用值填充，仅为保证指令长度。
- 数据存储指令：用于将数据（立即数）压入操作数栈。
格式：dstore x, x 为立即数。

虚拟机汇编——数据存取指令（出栈）

- 数据写入指令：用于将操作数栈顶的元素写入对应存储位置并将其弹出。
格式：dwrite x, x 为目标变量编码。
- 全局数据写入指令：用于将操作数栈顶的元素写入对应存储位置并将其弹出。
格式：dwrite x, x 为目标变量编码。
- 局部数据写入指令：用于将操作数栈顶的元素写入对应存储位置并将其弹出。
格式：dlwrite x, x 为目标变量编码。
- 绝对数据写入指令：需要从栈顶获取两个操作数，栈顶元素为地址，第二个元素为要写入的值，将值写入对应的内存地址并将它们弹出。
格式：dawrite x, x 为无用值填充，仅为保证指令长度。

函数调用的实现示例

- ① 调用者将函数参数依次压入操作数栈。
- ② 被调用者将函数参数一次性从操作数栈中弹出并存入对应的局部变量位置。
- ③ 被调用者执行完毕后将返回值（如果有）压入操作数栈。
- ④ 调用者从操作数栈中获取返回值用于后续操作。



虚拟机汇编——求值

- 求值指令：根据立即数 x 进行对应操作。
如果 x 对应二元运算，则取出操作数栈顶两个元素进行对应的计算后将结果存回操作数栈顶。
如果 x 对应条件跳转运算，则取出三个操作数，第一个元素（原栈顶）为表达式的结果。若表达式结果不为零，将 PC 设置为第二个元素（次栈顶，THEN 分支的地址），否则设置为第三个元素（再次栈顶，ELSE 分支的地址）。³
格式：eval x ， x 为立即数，运算类型与对应的值参考后面字节码介绍。

³编译器在生成代码时需要预先计算出 THEN 分支和 ELSE 分支的地址并将它们作为立即数通过 `dstore x` 压入栈中。

虚拟机汇编——跳转

- 直接跳转指令：该指令将直接把 PC 设置为 x 。
格式：jump x 。
- 链接跳转指令：记录当前的栈寄存器和 PC⁴，并根据栈顶元素申请内存，把栈寄存器设置为新的内存地址。然后将直接把 PC 设置为 x 。
格式：jal x 。
- 返回指令：从虚拟机控制的存储区中获取原来的栈寄存器和 PC 值并设置回去。⁵
格式：ret x ， x 忽略。
- 无操作指令：不执行任何操作。
格式：nop 0，后面的操作对象可以是任意整数，均应当被忽略。

⁴保存在由虚拟机管理的空间，对用户来说是透明的

⁵编译器生成代码时应当将生成 dstore、dload 等指令手动将函数返回值压入栈中，以供调用者获取。

虚拟机汇编——内存分配

- 内存分配指令：从操作数栈中获取一个元素作为需要分配的内存大小（单位为字节），申请对应大小的内存并将起始地址压入操作数栈。
格式：`malloc x`，`x` 为无用值填充，仅为保证指令长度。⁶

⁶对于字符串字面值而言，需要生成申请内存空间的指令以及逐字符复制的指令。此外，还需要额外处理存储内存地址的计算，如 `a[i] -> a + i * 4`。

虚拟机汇编——输入输出指令

- 整数输入指令：操作数无用。
格式：geti x，将从 stdin 读入一个十进制整数的数据存入栈顶。
- 字符输入指令：操作数无用。
格式：getc x，将从 stdin 读入一个字符并把它对应的 ASCII 值存入栈顶。
- 字符串输入指令：操作数无用。
格式：gets，将从 stdin 读入一个连续非空字符构成的字符串并把地址存入栈顶。
- 整数输出指令：操作数无用。
格式：puti x，将栈顶元素弹出并以十进制的形式输出到 stdout 中。
- 字符输出指令：操作数无用。
格式：putc x，将栈顶元素弹出转换为 UTF-8 字符输出到 stdout 中，否则什么都不做。
- 字符串输出指令：操作数无用。
格式：puts x，将栈顶元素弹出并判断它是否为合法的内存地址，若合法则从该地址开始按照 C 风格字符串要求转换为 UTF-8 编码后输出到 stdout 中，否则什么都不做。

EVAL 指令操作数

eval 指令操作数:

- 加法: 0x00010001
- 减法: 0x00010002
- 乘法: 0x00010003
- 整除: 0x00010004
- 求余: 0x00010005
- 大于: 0x00010006
- 小于: 0x00010007
- 大等于: 0x00010008
- 小等于: 0x00010009
- 等于: 0x0001000A
- 不等于: 0x0001000B
- 条件跳转: 0x0001000C